Practice 7

Objectives

- Map with JPA annotations the domain model developed in the previous lab session
- Mapping in *Green Field* and in *Brown Field* (*Legacy)*
- Adjust the implementation so that it can be mapped to the JPA specification
- Verify functionality and mapping with unit tests

About Carworkshop base

Get the initial skeleton

Download from the virtual campus the file ***ri.cws_base.jpa.2026.p7.skeleton.zip***. Uncompress it into a folder. It contains a maven root project with subprojects. Create a new workspace. If you use Eclipse import it using the option *Maven/Existing maven projects*.

This skeleton adds a new folder for the database.

Mapping the domain model generating the BDD

Initial Instructions:

- Copy your domain model classes developed in previous session to the `uo.ri.cws.domain` package.
- Reset the BDD with the reset script. Start up the HSQLDB database. It is empty, it has no tables.
- In the *persistence.xml* file **keep this line uncommented**:

```
<property name="eclipselink.ddl-generation" value="drop-and-create-tables"/>
```

This line causes the mapper to regenerate all tables each time it is loaded.

- In the domain classes annotate with `@Transient` all the attributes that are association ends.
- Add the *BaseEntity* class as the base class for all entities in the domain. With this, all entities will have:
  * UUID-based artificial identity (String)
  * A field *version* of type Long
  * Fields *createdAt* and *updatedAt* of type *LocalDateTime*
  * *hashCode()* and *equals()* based on the artificial id
- Annotate *BaseEntity* with `@MappedSuperclass`, `@Id` and `@Version`

We'll do incremental mapping, step by step:

- First, map the entities one by one, adding `@Entity` as appropriate.
  * After each annotated class, execute the *JustLoadEntityManagerFactory.java* supplied class. This forces the mapper to load the persistence unit and regenerate the tables.
  * Look at the tables generated with the database manager utility. They should appear as you write them down and execute the loader class.

- Next: Map the associations one by one and observe the changes in the tables by adding annotations at both ends (*@ManyToOne*, *@OneToMany*, etc.).
- Pay attention to the associations intra-aggregate. The solid diamonds in the UML class diagram indicate life cycle dependencies. That is supported in JPA with cascade effects and orphan removal. Complete the annotations properly.
- Map the inheritance of *PaymentMean* classes. Try the different strategies and observe the tables that are generated.
- Add annotations to entities so that the database adds a unique index to the natural identities of the entities. Use the annotations: `@Column(unique=true)` and/or `@Table(uniqueConstraints...)` depending on whether the natural identity is simple or compound.
- For monetary fields fix the precision and the scale at the database. Use a combination of `@AttributeOverride(…)` and `@Column(name = …, precision = 19, scale = 2))`
- Run the supplied tests as verification that the mapping is correct.

  Don't move to the next point until all tests pass without errors.

Mapping the domain model to an existing BDD schema

- With the database stopped, run the *4.setLegacy.bat* script. This loads the file containing the CarWorkshop database with data and names of tables and fields **different** from those generated by the mapper.
- **Comment** on the file `persistence.xml`. the line

```
<!--
<property name="eclipselink.ddl-generation" value="drop-and-create-tables"/>
-->
```

Now the mapper no longer regenerates the tables, so we must adapt the mapping to the tables they give us.

- There are these differences between the object model and the table model:
  * The tables follow the *T<Entity>s* naming pattern, e.g. *TCustomers*, *TVehicules*, etc.
  * The payment method hierarchy has a table for each class.
  * The enumerated type fields are in VARCHAR format in the tables (String and not Ordinal).
  * The names of the micro value types fields differ from the one defined in the class. You must use `@AttributeOverride`
- Make the necessary mapping adjustments so that the mapper can adapt to the new table schema and the tests pass correctly again.

About Carworkshop Extended (Delivery project)

Once you have all the previous tests running green then you can continue developing the delivery version. Download from the virtual campus the *ri.**cws_extended**.jpa.2026.**p7**.skeleton.zip*. It has the extended version of the database.

Import the new skeleton in *a new workspace* and then copy the domain model classes from the *cws_base* P7 project into the same package of the new project. Now you have to also map the extension of the domain model and make again all tests to pass. The /docs folder of this new project contains the new database table diagram.